

Software Requirements Specification (SRS)

RedBlueAI Safeguard — "RedAgent"

Document	Software Requirements Specification
Product	RedBlueAI Safeguard (RedAgent) — autonomous, non-destructive red-team validation agent
Version	1.0
Status	Baseline (reconstructed from implemented code, Phases 0–10)
Standard	Structured per IEEE Std 830 / ISO/IEC/IEEE 29148
Owner	ESDS — RedBlueAI Platform

Table of Contents

- 1. Introduction
- 2. Overall Description
- 3. External Interface Requirements
- 4. System Features & Functional Requirements
- 5. Non-Functional Requirements
- 6. Data Requirements
- 7. Compliance Requirements
- 8. Assumptions, Constraints & Traceability
- 9. Appendices

1. Introduction

1.1 Purpose

This SRS specifies the functional and non-functional requirements for **RedBlueAI Safeguard** (codename *RedAgent*), the red-team / purple-team loop of the RedBlueAI platform. Safeguard is an autonomous, safety-gated adversary-emulation agent that **non-destructively** simulates attacker behaviour against ESDS's authorised estate in order to **validate whether the existing defensive stack (SIEM, WAF, EDR, PAM, DAM) detects and responds** — then feeds the resulting gaps to the Detect and Act loops.

The intended audience is: platform engineers building/maintaining the agent, security operators running engagements, QA, security reviewers, and ESDS compliance stakeholders.

1.2 Scope

Safeguard **is**: - A breach-and-attack-simulation / continuous-security-validation engine. - A thin, safety-wrapped orchestration layer over standard open-source security tools (Nmap, Nuclei, Nikto, Prowler, Trivy, Simgrep, Gitleaks, Checkov, Dalfox, SQLMap, httpx, WhatWeb, Gobuster). - Driven by a planner (deterministic rule-based, or sovereign-LLM-based) over a checkpointed state machine, with human-in-the-loop approval on active steps. - The producer of a **detection-coverage matrix** and a **detection-gap report**.

Safeguard **is not**: - A mass-exploitation or offensive tool for third-party targets. - A destructive test tool — data-modifying, exfiltration, DoS, and persistence actions are disabled and unreachable. - A replacement for the operator's duty to test only authorised assets.

The defining question the product answers is not *"is this host vulnerable?"* but:

"When we perform technique T against asset A, does our Blue Team stack see it, alert on it, and stop it — and if not, why not?"

1.3 Definitions, Acronyms & Abbreviations

Term	Meaning
ROE	Rules of Engagement — signed scope/authorisation contract (<code>roe.yaml</code>)
Safety class	<code>passive</code> / <code>active-recon</code> / <code>active-validate</code> (<code>destructive</code> is disabled/unloadable)
HITL	Human-in-the-loop; a named approver signs off active steps
Detection Oracle	Component that asks the Blue Team stack whether an emulated action was noticed
Verdict	Per-action outcome: <code>BLOCKED</code> / <code>DETECTED</code> / <code>PARTIAL</code> / <code>MISSED</code>
MTTD	Mean/Measured Time To Detect (seconds from action to first detection)
Finding	A normalised vulnerability/observation from a tool
Validation	A non-destructive proof-of-signal outcome (gated, active)
Attack path	A candidate kill-chain of findings, ATT&CK-annotated
Gap report	List of every MISSED/PARTIAL with the expected-but-absent detection
Grounded token	A number/CVE sourced from a tool/DB artifact (not LLM-invented)
SIEM / WAF / EDR / PAM / DAM	Security Info & Event Mgmt / Web App Firewall / Endpoint Detection & Response / Privileged Access Mgmt (Nandi) / Database Activity Monitoring (Jatayoo)
CVE / CVSS / EPSS	Vulnerability ID / severity score / exploit-probability score
NVD / ATT&CK	National Vulnerability Database mirror / MITRE ATT&CK technique taxonomy
DPDP / CERT-In	India Digital Personal Data Protection Act / Indian Computer Emergency Response Team

1.4 References

- `README.md` , `docs/ARCHITECTURE.md` , `docs/WORKFLOW.md` , `docs/ROADMAP.md` , `docs/SAFETY.md` , `IMPLEMENTATION.md` , `docs/HLD_LLD.md` .

- Config contracts: `roe.example.yaml`, `tools.yaml`, `settings.example.yaml`.
- MITRE ATT&CK; NIST NVD; FIRST EPSS; IEEE 830; ISO/IEC/IEEE 29148.

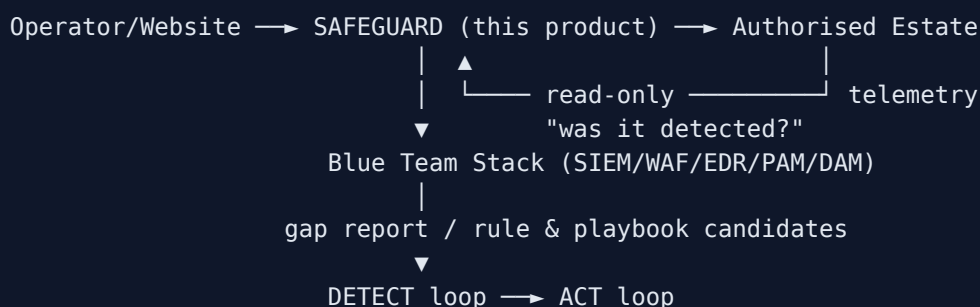
1.5 Overview

Section 2 gives the overall product context and constraints; Section 3 the external interfaces; Section 4 the functional requirements (grouped by system feature, each requirement carrying an **FR-** identifier); Section 5 the non-functional requirements (**NFR-**); Sections 6–8 data, compliance, and traceability.

2. Overall Description

2.1 Product Perspective

Safeguard is one of three cooperating loops. It is the **only** loop that intentionally generates attacker activity, precisely so Detect and Act can be measured against ground truth.



Internally the product is composed of: a Control Plane / API, a planner-driven orchestration graph, a cross-cutting safety layer, a tool-adaptor framework with a sandbox runner, an offline intelligence subsystem, the Detection Oracle, and a reporting/handoff subsystem. (See [docs/HLD_LLD.md](#) for component and class detail.)

2.2 Product Functions (summary)

- **F1** Attack-surface discovery (recon): Nmap, httpx, WhatWeb, Gobuster.
- **F2** Vulnerability detection (scan): Nuclei, Nikto, Trivy, Prowler; SAST/ secrets/IaC (Semgrep, Gitleaks, Checkov) for white-box.
- **F3** Safe, non-destructive validation (gated): Dalfox reflection, SQLMap detection-only.
- **F4** Intelligence & correlation: local NVD CVE enrichment, ATT&CK mapping, risk scoring, attack-path correlation.
- **F5** Detection Oracle: per-action detection verdict + MTTD, coverage matrix, gap report.
- **F6** Reporting & evidence: technical/executive reports, ATT&CK heatmap, detection-gap report, content-addressed evidence.
- **F7** Continuous mode & integration: baseline diffing / regression detection; Detect/Act handoff.
- **F8** Safety & governance: scope/window/approval gates, rate limiting, kill switch, immutable audit, RBAC.
- **F9** Control plane / API: engagement lifecycle, approvals, mode contract.

2.3 User Classes and Characteristics

User class	Description	Privileges (RBAC role)
Administrator	Platform owner	start, approve, kill, query (<code>admin</code>)
Operator	Runs engagements	start, kill, query (<code>operator</code>)
Approver	Named human who signs off active steps; must be listed in the ROE	approve, query (<code>approver</code>)
Viewer	SOC analyst / stakeholder reading reports	query (<code>viewer</code>)
Website / API client	Automated caller submitting scans over HTTP	scoped to scan submission
Downstream loops	Detect & Act consume the handoff artifacts	read handoff

2.4 Operating Environment

- **Runtime:** Python ≥ 3.10 on Linux; sovereign ESDS cloud (India-resident).
- **Inference:** Qwen served on ESDS sovereign GPU via an OpenAI-compatible endpoint (`SAFEGUARD_LLM_*`). Deterministic rule planner works fully offline.
- **Sandbox (target):** gVisor/Firecracker egress-pinned microVMs; dev uses a local subprocess runner.
- **Storage:** SQLite checkpoints, JSONL audit/handoff, content-addressed evidence files, JSON reports/baselines.
- **Data sources:** local NVD mirror + local ATT&CK keyword map (no outbound NVD/MITRE calls at runtime).

2.5 Design & Implementation Constraints

- **C1** No foreign/external API dependency in the default build (sovereignty).
- **C2** Minimal dependency surface — safety core uses only the standard library; HTTP API adds Starlette + uvicorn only.
- **C3** The LLM must never execute tools; it may only propose.
- **C4** The `destructive` safety class must not be representable or loadable.
- **C5** All security-relevant checks must fail closed.
- **C6** Secrets must come from the environment / sovereign secret store, never from config files or model output.
- **C7** Clocks are injected so the safety core is deterministic and testable.

2.6 Assumptions & Dependencies

- **A1** A signed, valid ROE (with `authorisation_ref` and ≥ 1 approver) exists per engagement.
- **A2** Oracle connectors have dedicated **read-only** service accounts to the Blue Team stack.
- **A3** Required scanner binaries are installed on the runner's PATH (missing tools are reported, not silently skipped).
- **A4** The estate under test is owned by or contractually authorised to ESDS.

3. External Interface Requirements

3.1 User Interfaces

- **UI-1 Command line** (`safeguard`): subcommands `run` (phase recon|scan), `engage` (planner-driven full engagement), `scope-check` (test a target against the ROE), `audit-verify` (verify the on-disk hash chain).
- **UI-2 HTTP/JSON API** (website integration): async scan jobs with polling.
- **UI-3 Report bundle**: human-readable Markdown (executive summary, technical report, ATT&CK heatmap, gap report) plus machine-readable `report.json`.

3.2 Hardware Interfaces

- **HW-1** GPU host for sovereign LLM inference (production).
- **HW-2** Container/microVM host for the egress-pinned sandbox runner. No direct interface to specialised hardware beyond standard compute/network.

3.3 Software Interfaces

- **SW-1 LLM**: `POST {base_url}/v1/chat/completions` (OpenAI-compatible), bearer key from env; Qwen hybrid-thinking toggle via `chat_template_kwargs`.
- **SW-2 Security tools**: 13 CLI binaries invoked via adapters as subprocesses.
- **SW-3 Detection telemetry**: read-only `TelemetryBackend.query(target, start, end, technique)` contract (Wazuh/WAF/EDR/PAM/DAM behind it).
- **SW-4 Intel data**: local NVD JSON mirror; local ATT&CK keyword YAML.
- **SW-5 Detect/Act**: JSON handoff files (production: POST to Detect/Act APIs).

3.4 Communication Interfaces

- **CM-1** HTTP/JSON for the API; CORS defaults to loopback origins, override via `SAFEGUARD_API_CORS_ORIGINS`.
 - **CM-2** HTTPS to the sovereign LLM endpoint.
 - **CM-3** Sandbox egress restricted to the ROE allowlist (target deployment).
-

4. System Features & Functional Requirements

Priority key: **M** = Mandatory, **S** = Should-have, **C** = Could-have. "Implemented" reflects the current codebase; "Deployment" items are behind an existing interface seam.

4.1 Feature — Rules of Engagement & Scope Management

- **FR-1 (M)** The system shall load a typed ROE from YAML defining: engagement id, owner, authoriser, `authorisation_ref`, mode, profile, in-scope (domains/CIDRs/cloud accounts/repos), exclusions (hosts/paths), timezone, time windows, named approvers, and budget.
- **FR-2 (M)** The system shall **reject at load time** any ROE lacking an `authorisation_ref`, using a profile other than `non_destructive`, or naming no approver (fail closed).
- **FR-3 (M)** The system shall validate every target against the ROE allowlist before any packet is sent: single IPs against CIDRs; a CIDR target only if it is a subset of an allowed network; domains by exact or parent-suffix match; repos and cloud accounts literally.
- **FR-4 (M)** Exclusions (hosts, IPs, URL path globs) shall override the allowlist unconditionally.
- **FR-5 (M)** The system shall decompose URL targets into host + path, matching the host against scope and the path against exclusions.
- **FR-6 (M)** The system shall deny any action outside the permitted time window(s) in the ROE timezone, and fail closed if no window is defined.
- **FR-7 (M)** A `scope-check` command shall report whether a given target is in scope, excluded, or out of scope, without executing any tool.

4.2 Feature — Safety Pipeline & Guardrails

- **FR-8 (M)** Every tool invocation shall pass, in order, through: kill-switch check → scope check → window check → safety-class/approval check → command build + forbidden-flag validation → global destructive-token profile guard → rate limiter → sandbox execution → output parse → evidence capture → audit.
- **FR-9 (M)** A tool adapter shall be executable **only** from within the safety pipeline; adapters shall not be able to execute themselves.
- **FR-10 (M)** Any gate failure shall raise a typed `SafetyViolation`, be audited, and return `allowed=False` with no tool execution.
- **FR-11 (M)** The system shall enforce a four-layer defence against destructive actions: (a) the `destructive` class is not representable/loadable; (b) per-tool `forbidden_flags`; (c) per-adapter mode ceilings; (d) a global profile-guard token denylist applied to every command.
- **FR-12 (M)** The system shall enforce per-target request-rate and concurrency ceilings plus a global engagement action budget; exceeding any shall throttle/ deny without mutating limiter state.
- **FR-13 (M)** A single kill-switch action shall halt in-flight work and revoke the sandbox runner.
- **FR-14 (S)** In the target deployment, the sandbox shall be egress-pinned to the ROE allowlist so a tool physically cannot reach an out-of-scope host.

4.3 Feature — Recon & Asset Discovery

- **FR-15 (M)** The system shall perform active recon (Nmap service/version, httpx live-host + tech, WhatWeb fingerprint) against in-scope targets.
- **FR-16 (S)** The system shall support content/directory discovery (Gobuster) given a wordlist.
- **FR-17 (M)** The system shall normalise heterogeneous tool output into a unified `Asset` model and deduplicate/merge assets on `(type, address, port, protocol)`, deep-merging technology fingerprints across tools.

- **FR-18 (M)** Recon step denials or errors shall be recorded and shall not abort the engagement.
- **FR-19 (C)** (*Deferred*) Subdomain enumeration (subfinder/amass) — not yet implemented; slots into the recon flow when added.

4.4 Feature — Vulnerability Detection (Scan)

- **FR-20 (M)** The system shall detect vulnerabilities via Nuclei and Nikto (black-box), Prowler and Trivy (gray-box cloud), and Semgrep/Gitleaks/Checkov/ Trivy (white-box source).
- **FR-21 (M)** Nuclei shall run under a safe-only template policy: intrusive/DoS/ fuzz/brute-force tags excluded, and the adapter shall block re-enabling them.
- **FR-22 (M)** The system shall normalise findings into a unified **Finding** model and deduplicate across tools on **(asset_ref, title)** , keeping the highest severity and unioning CVEs/evidence/techniques.
- **FR-23 (M)** Gitleaks findings shall never store the secret value (DPDP).
- **FR-24 (M)** Cloud (Prowler) scanning shall be read-only; mutating/remediation flags shall be rejected.

4.5 Feature — Planning & Orchestration

- **FR-25 (M)** A planner shall decide the next phase and return a **PlanDecision** only; it shall never execute anything.
- **FR-26 (M)** The system shall provide a deterministic **RulePlanner** (recon → scan → correlate → optional gated validate → report) as the offline default.
- **FR-27 (S)** The system shall provide an **LLMPlanner** that obtains a structured JSON decision from the sovereign LLM and **validates it in code** against the tool registry and safety classes; an unknown or non- **active-validate** tool proposal shall be downgraded to **report** .
- **FR-28 (M)** The LLM planner shall not be able to widen its own authority (propose an unregistered tool, a destructive action, or a class it is not permitted).
- **FR-29 (M)** The LLM planner shall include anti-loop and anti-premature-report guards (do not repeat one-shot stages; do not report a live surface before recon+scan).
- **FR-30 (M)** The orchestration shall be a checkpointed state machine whose state is persisted after every node, enabling resume and replay.
- **FR-31 (S)** If the LLM is unconfigured or fails, the system shall fall back to the deterministic planner and record that fact explicitly.

4.6 Feature — Human-in-the-Loop Approval

- **FR-32 (M)** Any **active-validate** step shall pause the engagement at an interrupt and create an approval request, persisting until resolved.
- **FR-33 (M)** Only a human named in the ROE **approvers** list shall be able to approve an active step; approval by a non-named party shall be rejected.
- **FR-34 (M)** On approval the engagement shall resume exactly where it parked; on denial the step shall be skipped and control returned to the planner.
- **FR-35 (S)** Approval requests should support a timeout that, on expiry, skips the step and audits the event. (*Not yet implemented.*)

4.7 Feature — Safe Validation

- **FR-36 (M)** The system shall confirm the *signal* of a finding non-destructively only: reflected-XSS confirmation (Dalfox, benign marker) and boolean/time SQLi **detection** (SQLMap, no dump/shell/enumeration).
- **FR-37 (M)** Validation shall run only after approval, only in the `non_destructive` profile, and shall stamp the approving operator onto the `Validation` record.
- **FR-38 (M)** Each validation shall capture content-addressed evidence and carry a stable evidence reference.

4.8 Feature — Intelligence & Correlation

- **FR-39 (M)** The system shall enrich findings with CVE detail from a **local** NVD mirror (no external NVD calls at runtime).
- **FR-40 (M)** The system shall map findings/actions to MITRE ATT&CK techniques using a local (offline) mapping.
- **FR-41 (M)** The system shall compute risk from CVSS + EPSS + asset criticality + **detection status**, such that an undetected medium can outrank a detected high.
- **FR-42 (S)** The system shall correlate findings per asset into candidate attack paths ordered along the ATT&CK tactic sequence, each step carrying its detection verdict.
- **FR-43 (M)** A numeric-claim verifier shall flag any CVE in report narrative not present in the grounded-token set; ungrounded figures shall be reported.

4.9 Feature — Detection Oracle ★

- **FR-44 (M)** After **every** emulated action (recon/scan/validate), the system shall query all read-only detection connectors over the action's time window and target.
- **FR-45 (M)** The system shall score a single verdict per action — `BLOCKED` / `DETECTED` / `PARTIAL` / `MISSED` (best across sources) — plus MTTD (earliest detection relative to action time).
- **FR-46 (M)** The system shall record detection results into engagement state and maintain a per-target detection-status map consumed by risk scoring.
- **FR-47 (M)** The system shall produce a coverage matrix (coverage %, mean MTTD, per-technique matrix) and a gap report listing every MISSED/PARTIAL with the expected detection.
- **FR-48 (S)** Detection connectors shall be read-only by construction; only WAF/ EDR may return `BLOCKED`.

4.10 Feature — Reporting & Evidence

- **FR-49 (M)** The system shall generate a report bundle per engagement: `report.json`, executive summary, technical report, ATT&CK coverage heatmap, and detection-gap report, written to `runs/<id>/report/`.
- **FR-50 (M)** All raw tool output shall be captured as content-addressed evidence, referenced by hash from findings and validations.
- **FR-51 (M)** Report narrative figures shall be grounded; the executive summary shall flag any ungrounded figure.

- **FR-52 (M)** The report shall emit a `detection_index` (technique|host → verdict) to support regression diffing.

4.11 Feature — Continuous Mode & Platform Integration

- **FR-53 (S)** The system shall store report bundles as per-engagement baselines and diff a fresh run against the prior baseline.
- **FR-54 (S)** The diff shall surface **regressions** (a technique/host that was DETECTED/BLOCKED and is now MISSED/PARTIAL), improvements, new/resolved gaps and findings, and coverage delta.
- **FR-55 (S)** The system shall convert detection gaps into SIEM/WAF **rule candidates** for the Detect loop and candidate **response playbooks + tickets** for the Act loop, written to `runs/<id>/handoff/`.
- **FR-56 (C)** (*Deployment*) Continuous scheduling per asset group and delivery of handoff artifacts to live Detect/Act APIs.

4.12 Feature — Control Plane & API

- **FR-57 (M)** The system shall expose a self-describing mode contract (required/ optional inputs, tools, examples) at `GET /api/v1/modes`.
- **FR-58 (M)** The system shall accept a scan request (`POST /api/v1/scans`), validate mode-specific inputs (fail closed), run it as a background job, and return a job id for polling.
- **FR-59 (M)** The API shall synthesise a per-request non-destructive ROE; the scope guard shall still fail closed on any target outside the synthesised scope.
- **FR-60 (M)** The API shall report unavailable scanner binaries so a zero-finding scan caused by a missing tool is explained rather than mistaken for a clean target.
- **FR-61 (M)** Control-plane actions (start/approve/kill/query) shall be RBAC role-gated and audited per operator.
- **FR-62 (M)** The audit chain shall be independently verifiable (`audit-verify`), reporting INTACT/BROKEN and the event count.

4.13 Feature — Auditability

- **FR-63 (M)** Every plan decision, tool proposal, approval event, tool execution, result, and denial shall be appended to an immutable, hash-chained, append-only audit log.
- **FR-64 (M)** Tampering with any audit record shall be detectable by recomputing the chain.

5. Non-Functional Requirements

5.1 Safety (highest priority)

- **NFR-1** All scope, window, approval, and profile checks shall **fail closed**: any ambiguity denies the action.
- **NFR-2** No supported configuration shall permit a destructive, out-of-scope, or out-of-window action simultaneously; the destructive class is unreachable.

- **NFR-3** Untrusted content (scanned pages, source, tool output) shall never be treated as executable instructions (prompt-injection resistance): the model cannot execute, and gates sit between any proposal and execution.

5.2 Security

- **NFR-4** Secrets (LLM key, connector creds) shall be sourced only from the environment / sovereign secret store, never from config files or model output.
- **NFR-5** Detection Oracle connectors shall use least-privilege read-only accounts.
- **NFR-6** Control-plane actions shall be role-gated (RBAC) and per-operator audited.
- **NFR-7** Safeguard shall run in its own isolated tenant, separate from the production workloads it tests.

5.3 Sovereignty & Data Residency

- **NFR-8** The default build shall have zero foreign API dependencies (structural, not a config toggle).
- **NFR-9** CVE data (local NVD mirror), ATT&CK data (local bundle), evidence, and audit logs shall be India-resident on ESDS infrastructure.

5.4 Performance & Scalability

- **NFR-10** A bounded black-box web scan (single host) should complete within ~1–2 minutes (tuned Nuclei profile; Nikto `-maxtime 120`).
- **NFR-11** The LLM client shall tolerate serverless cold starts (≥ 180 s timeout) and degrade gracefully to the rule planner on failure.
- **NFR-12** The API shall run scans as background jobs so requests return immediately with a poll handle.
- **NFR-13** (*Deployment*) The job store and checkpointing shall be swappable for durable/scalable backends (Redis/DB, Postgres) without changing call sites.

5.5 Reliability & Recoverability

- **NFR-14** Engagement state shall be checkpointed after every node so a run is resumable and replayable.
- **NFR-15** A sandbox/parse failure shall be caught and recorded as an error outcome without crashing the engagement.

5.6 Maintainability & Portability

- **NFR-16** Adding a new tool shall require only a `tools.yaml` entry plus an adapter, with no framework changes.
- **NFR-17** Dev/production backends (graph engine, sandbox runner, telemetry backend, planner) shall each sit behind a stable interface so swaps are wiring changes, not rewrites.
- **NFR-18** The safety core shall depend only on the Python standard library.

5.7 Testability

- **NFR-19** All time-dependent components shall accept injected clocks so runs are deterministic and unit-testable offline.
- **NFR-20** The system shall ship an automated test suite covering the safety gates, adapters, planner, oracle, reporting, and integration (~96 tests).

5.8 Observability

- **NFR-21** The system shall expose counters/gauges (dependency-free) suitable for export to Prometheus/OTel in deployment.

6. Data Requirements

6.1 Core entities (persisted/normalised)

Entity	Key fields
RulesOfEngagement	id, owner, authoriser, authorisation_ref, mode, profile, scope, exclusions, timezone, windows, approvers, budget
Asset	id, address, type (host/service/endpoint/repo/cloud-account), port, protocol, service, tech, in_scope
Finding	id, title, asset_ref, source_tool, severity, cve_ids[], cvss, epss, attack_techniques[], evidence_refs[], status
Validation	target, method, result (confirmed/inconclusive), tool, approved_by, evidence_ref, non_destructive=true
DetectionResult	action_ref, target, technique, verdict, source, rule_id, ttd_seconds
AttackPath	asset, steps[] (finding → technique → tactic → detection verdict → risk), overall_risk
AuditEvent	seq, ts, actor, action, engagement_id, params_hash, detail, prev_hash, hash
RegressionReport	regressions[], improvements[], new/resolved gaps & findings, coverage_delta

6.2 On-disk layout (per engagement)

```
runs-*/<engagement-id>/
  audit.log.jsonl · checkpoints.db · roe.generated.yaml
  evidence/ev-<hash>.txt
  baselines/baseline-*.json
  report/{report.json, executive_summary.md, technical_report.md,
          attack_heatmap.md, detection_gap_report.md}
  handoff/{detect_rule_candidates.json, act_playbooks.json, act_tickets.json}
```

6.3 Data handling constraints

- **DR-1** Secret values discovered (Gitleaks) shall not be persisted.
 - **DR-2** Evidence capture shall exclude personal-data payload content by default (DPDP).
 - **DR-3** The `DetectionResult` set is the primary product; a `MISSED` finding carries more value to Detect/Act than a detected critical.
-

7. Compliance Requirements

- **CR-1 (CERT-In)** The system shall maintain a complete, timestamped audit trail of its own actions supporting directed security-testing and reporting expectations.
- **CR-2 (DPDP Act)** Testing shall be non-destructive and read-only with respect to data; the agent shall not exfiltrate or copy personal data.
- **CR-3 (RBI / sector frameworks)** Continuous control validation and coverage reporting shall support periodic control-effectiveness assessment for regulated workloads.
- **CR-4 (ISO 27001 / SOC 2)** Engagement reports and evidence bundles shall be structured to serve as control-effectiveness evidence.

Exact obligations to be confirmed with ESDS compliance before customer-facing use; the above states design intent, not legal advice.

8. Assumptions, Constraints & Traceability

8.1 Known limitations (current baseline)

- Local subprocess runner is **not** egress-pinned (dev only); production sandbox is a deployment task.
- Detection connectors run against an in-memory telemetry backend in dev; live read-only Wazuh/WAF/EDR/PAM/DAM is a deployment task.
- The numeric-claim verifier currently grounds CVE IDs (CVSS/EPSS/counts/ports grounding is a planned extension).
- Approval timeout, subdomain enumeration, a distinct `enumerate` phase, real Detect/Act API delivery, and continuous scheduling are not yet implemented.
- The HTTP scan API auto-signs-off active-validate steps and has no authentication; the RBAC control plane is not yet exposed over HTTP.

8.2 Requirement ↔ phase traceability

Feature (§4)	ROADMAP phase	Primary modules
ROE & scope (4.1)	P0	<code>config/</code> , <code>safety/scope_guard.py</code>
Safety pipeline (4.2)	P0–P1	<code>safety/pipeline.py</code> , <code>rate_limiter</code> , <code>killswitch</code> , <code>profile</code>
Recon (4.3)	P2	<code>recon/</code> , adapters <code>nmap/httpx/whatweb/gobuster</code>
Scan (4.4)	P3, P9	<code>scan/</code> , adapters <code>nuclei/nikto/trivy/prowler/semgrep/gitleaks/checkov</code>
Planning (4.5)	P4	<code>graph/</code> , <code>llm/planner.py</code> , <code>orchestrator.py</code>
Approval (4.6)	P4	<code>safety/approvals.py</code> , <code>graph/build.py</code>
Validation (4.7)	P5	<code>validate/flow.py</code> , adapters <code>dalfox/sqlmap</code> , <code>evidence.py</code>
Intelligence (4.8)	P6	<code>intel/</code>
Detection Oracle (4.9)	P7	<code>oracle/</code>
Reporting (4.10)	P8	<code>reporting/</code> , <code>evidence.py</code>
Continuous & integration (4.11)	P10	<code>continuous/</code> , <code>integration/</code>
Control plane & API (4.12)	P0/P10	<code>api/</code> , <code>safety/rbac.py</code>
Auditability (4.13)	P0	<code>safety/audit.py</code>

9. Appendices

Appendix A — Safety classes & gates

Class	Gate	Example tools
<code>passive</code>	rate limit	<code>cve_lookup</code> , <code>semgrep</code> , <code>gitleaks</code> , <code>checkov</code>
<code>active-recon</code>	rate limit	<code>nmap</code> , <code>httpx</code> , <code>whatweb</code> , <code>gobuster</code> , <code>nuclei</code> , <code>nikto</code> , <code>trivy</code> , <code>prowler</code>
<code>active-validate</code>	approval + rate limit	<code>dalfox</code> , <code>sqlmap</code>
<code>destructive</code>	not loadable / unreachable	—

Appendix B — Verdict precedence

`BLOCKED` > `DETECTED` > `PARTIAL` > `MISSED` (best across all detection sources).

Appendix C — Operating modes

Mode	Required input	Scanners
black_box	URL / host[:port]	nmap → nuclei, nikto
gray_box	cloud account IDs	prowler, trivy
white_box	local source paths	semgrep, gitleaks, checkov, trivy (no recon)

Appendix D — Environment configuration (non-secret examples)

```
SAFEGUARD_LLM_MODEL, SAFEGUARD_LLM_BASE_URL, SAFEGUARD_LLM_API_KEY (from secret store)
SAFEGUARD_MODE, SAFEGUARD_PROFILE=non_destructive
SAFEGUARD_API_PORT, SAFEGUARD_API_CORS_ORIGINS
SAFEGUARD_WAZUH_URL, SAFEGUARD_WAF_LOG_SOURCE, ... (read-only connector endpoints)
```

End of SRS. This specification is traceable to the implemented Phases 0–10; items marked (Deployment) sit behind interfaces already present in the codebase.